Imports

```
# import the necessary packages
import numpy as np
from dataclasses import dataclass, asdict
```

Game Setup

```
# Define the game state using a dataclass
@dataclass
class RandomGrid():
    state: 'typing.Any' = None  # The current grid state
    to_move: int = 0            # Which player's turn it is: 0 = MAX, 1 = MIN
    num_players: int = 2        # Number of players

    # Creates a new instance of the game with updated attributes
    def update(self, **updates):
        return self.__class__(**{**asdict(self), **updates})

    # Advances to the next player's turn
    def end_turn(self):
        return self.update(to_move=(self.to_move + 1) % self.num_players)

    # Initializes the game with a random 4x4 grid of integers between 0 and 9
    def random_init(self):
        return self.update(state=np.random.randint(0,10,(8,8)))

    # The game ends when the grid is reduced to a single cell
    def is_terminal(self):
        return len(self.state) == 1

    # Returns the utility of the terminal state
    def get_utility(self):
        return self.state[0,0] if self.is_terminal() else None

    # Returns a dictionary of valid moves for the current player
    def get_moves(self):
        if self.to_move == 0:  # MAX player's turn
            return {
                'L': choose_left,
                'R': choose_right
            }
        else:  # MIN player's turn
            return {
                'T': choose_top,
                'B': choose_bottom
            }

# Applies the selected move and ends the turn
def make_move(game, move):
    return move(game).end_turn()

# MAX player move - reduces the grid to the left half
def choose_left(game):
    s = game.state
    return game.update(state=s[:,:int(s.shape[1]/2)])

# MAX player move - reduces the grid to the right half
def choose_right(game):
    s = game.state
    return game.update(state=s[:,int(s.shape[1]/2):])

# MIN player move - reduces the grid to the top half
def choose_top(game):
    s = game.state
    return game.update(state=s[:int(s.shape[0]/2),:])

# MIN player move - reduces the grid to the bottom half
def choose_bottom(game):
    s = game.state
    return game.update(state=s[int(s.shape[0]/2):,:])
```

Minimax

```
# Returns the optimal minimax value for the current player
def minimax(game):
    """
    Entry point to minimax search.
    If it's MAX's turn, return the result of max_value.
    If it's MIN's turn, return the result of min_value.
    """
    if game.to_move == 0:
        return max_value(game)
    else:
        return min_value(game)

# Evaluates the best possible outcome for the MAX player
def max_value(game):
    """
    Return the highest utility value MAX can guarantee from this state,
    along with the move that leads to it.
    """
    if game.is_terminal():
        return game.get_utility(), None

    best_value, best_move = -np.inf, None
    for move_name, move in game.get_moves().items():
        value, _ = min_value(make_move(game, move))
        if value > best_value:
            best_value, best_move = value, move_name

    return best_value, best_move

# Evaluates the best possible outcome for the MIN player
def min_value(game):
    """
    Return the lowest utility value MIN can guarantee from this state,
    along with the move that leads to it.
    """
    if game.is_terminal():
        return game.get_utility(), None

    best_value, best_move = np.inf, None
    for move_name, move in game.get_moves().items():
        value, _ = max_value(make_move(game, move))
        if value < best_value:
            best_value, best_move = value, move_name

    return best_value, best_move
```

Run Game

```
# Main game loop
def play(game):
    optimal_score = minimax(game)[0]  # Evaluate optimal score
    while not game.is_terminal():
        to_move = game.to_move
        valid_input = False

        while not valid_input:
            allowable_inputs = [m for m in game.get_moves()]

            print(game.state)
            print(f"Player {'MAX' if to_move == 0 else 'MIN'} to move")
            print("Optimal move, ", minimax(game))

            player_input = input(f"Options: {', '.join(allowable_inputs)}\n")
            if player_input in allowable_inputs:
                valid_input = True
            else:
                print(f"Please input one of the following: {', '.join(allowable_inputs)}")

        # Apply the move and switch to the other player
        game = make_move(game, game.get_moves()[player_input])
```

```
        print()

    # End of game
    print(f"The outcome of the game is {game.get_utility()}")
    print(f"The optimal play is {optimal_score}") # Reveal optimal score

# Start game with a random grid
game = RandomGrid().random_init()
play(game)
```

```
[[9 7 7 0 9 7 3 6]
 [5 2 3 4 9 4 4 7]
 [3 6 9 5 6 9 4 4]
 [7 7 0 9 9 0 4 3]
 [1 8 6 6 3 6 5 5]
 [0 3 8 4 4 7 1 3]
 [6 1 8 5 9 9 5 5]
 [4 8 8 3 3 7 4 3]]
Player MAX to move
Optimal move,  (np.int64(6), 'R')

[[9 7 3 6]
 [9 4 4 7]
 [6 9 4 4]
 [9 0 4 3]
 [3 6 5 5]
 [4 7 1 3]
 [9 9 5 5]
 [3 7 4 3]]
Player MIN to move
Optimal move,  (np.int64(6), 'T')

[[9 7 3 6]
 [9 4 4 7]
 [6 9 4 4]
 [9 0 4 3]]
Player MAX to move
Optimal move,  (np.int64(6), 'L')
```